



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Nicolás Fernández Tenreiro
09/05/2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

The project aimed to predict the success of SpaceX rocket landing using historical launch data. Different machine-learning models were evaluated including Logistic Regression, Support Vector Machines, Decision Trees and K-nearest neighbors.

Executive Summary

- **Summary of methodologies**

The project was made using different evaluation models trying to approach the one than better fits our principal objective of predict the landing of the rocket launches:

- **Logistic Regression.** Based in linear classification.
- **Support Vector Machine.** Captures non-linear boundaries using a function (kernel) than allow us to separate the data in groups than are not possible to separate linearly.
- **Decision Tree.** Interpretable model than allow to split data in different nodes based in different explicit and defined rules.
- **K-nearest neighbors.** Distance based classifier that predict labels by looking at the closest training samples.

Executive Summary

- **Summary of methodologies**

Optimization of the models:

We use GridSearchCV with 10 folds to crossvalidate our models.

Parameters than we explore:

C, kernel, gamma, max_depth, max_features, min_samples_leaf, min_samples_split, L1 “Lasso”, L2 “Ridge”, the algorithm of optimization LBFGS, n_neighbors, algorithm and p (used to control distances).

Metrics used to compare the models:

Confussion Matrix and accuracy.

Comparative results:

The model that outperforms all the others is the Decision Tree classifier. It achieves the highest predictive accuracy (around 88.89%) correctly identifying both successful landings and failed landings more reliably than the rest.

Executive Summary

- **Summary of all results**

After the evaluation of the distinct models, the Decision Tree delivered the best performance, achieving an accuracy close to 88,89% on the dataset. This model provides the most reliable predictions for identifying both successful and failed landings, supporting more informed decision-making in launch planning.

Introduction

- **Project background and context**

SpaceX is a disruptive aerospace company that transform the industry by introducing pioneering reusable rockets, this means a significantly reducing of launch costs. However, landing a rocket depends of the sum of multiple variables, including launch conditions, mission type, booster characteristics and environmental factors. Understanding how these variables interact is esencial to improve reliability and future mission planning.

The dataset used in this project contains detailed information about SpaceX launches, incluying launch sites, payload mass, booster versions, flight numbers and landing results.

Introduction

- **Problems that I want to find the answers:**
 - What factors influence the most in the SpaceX rocket lands successful?
 - Can machine learnings models predict accurately if a launch outcome based on historical data?
 - Which model performs best for this classification task, and why?
 - How can these predictions support decision-making and operational improvements?

Section 1

Methodology

Methodology

Executive Summary

- **Data collection methodology:**
 - Data was collected from two primary sources:
 - The SpaceX REST API, which provides structured launch data.
 - Web scraping of the SpaceX Wikipedia page to extract additional launch details.
- **Perform data wrangling**
 - Cleaned missing values, standardized columns formats, and merging datasets from API and scraping.
 - Engineered new features such as Class that classifies the landing successful or not outcome, labels and booster categories.
 - Converted raw Json and HTML data into structured Dataframes.
 - Normalized nested API fields.
 - Remove duplicates and irrelevant columns.

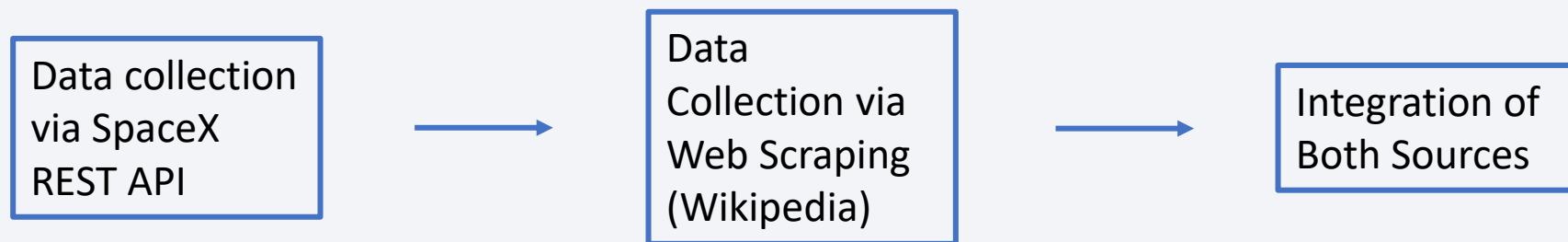
Methodology

Executive Summary

- Perform exploratory data analysis (EDA) using visualization and SQL
 - Visualized launch success rates, payload distributions, and site performances.
 - Used SQL queries to explore unique launch sites, better understand the payload mass variable, know the total successful and failure mission outcomes, etc.
- Perform interactive visual analytics using Folium and Plotly Dash
 - Built a Folium map showing launch sites, landing outcomes, and payload ranges.
 - Developed a Plotly Dash dashboard with interactive filters and dynamic charts.
- Perform predictive analysis using classification models
 - Trained and tuned classification models: Logistic Regression, SVM, KNN, and Decision Tree.
 - Evaluated models using 10-fold cross-validation and accuracy metrics.
 - Used GridSearchCV to optimize hyperparameters.
 - Compared validation accuracy and confusion matrices to select the best model.

Data Collection

The dataset used in this project was built by combining two complementary data-collection methods: structured retrieval through the SpaceX REST API and unstructured extraction through web scraping of the Falcon 9 launch records on Wikipedia. Both sources were cleaned, standardized, and merged to create a complete and reliable dataset for analysis.



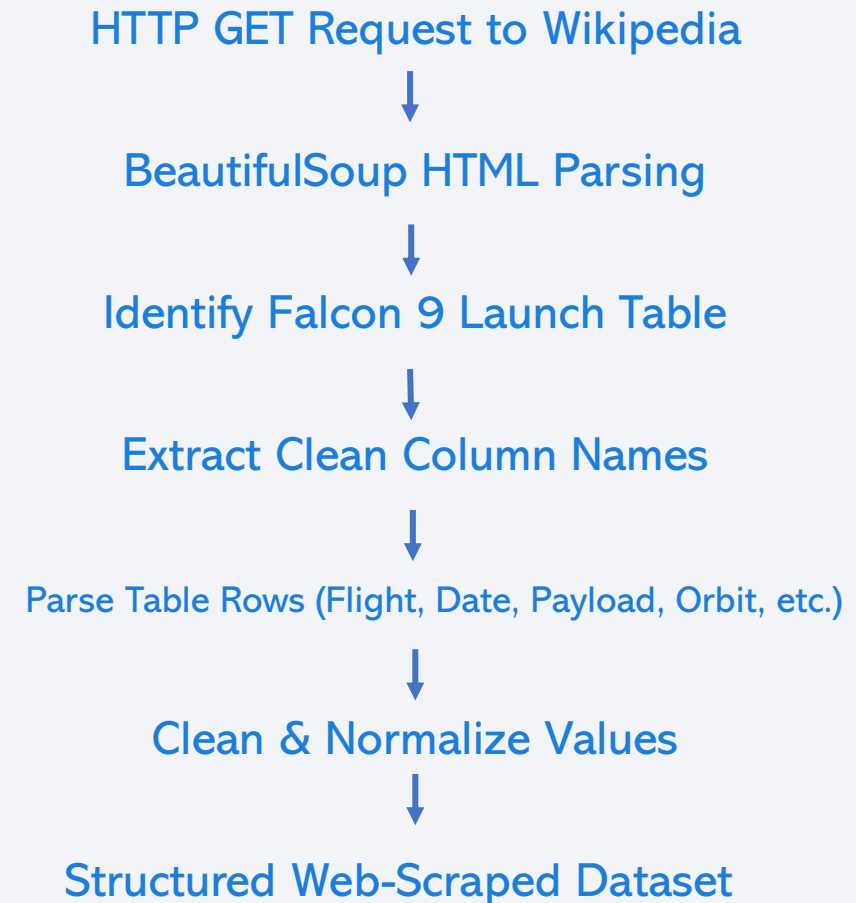
Data Collection – SpaceX API

- Retrieved historical launch data using the SpaceX REST API.
- Performed HTML GET requests to obtain all past launch records.
- Converted the JSON response into a structured Dataframe using JSON normalization.
- Filtered out launches with multiple cores or payloads to ensure consistency.
- Extracted key identifiers: rocket ID, payload ID, launchpad ID, core ID.
- Performed additional API calls to enrich the dataset with:
 - Booster version.
 - Launch site name + coordinates.
 - Payload mass + orbit.
 - Core attributes (block, reuse count, landing success, landing type, gridfins, legs, etc)
- Built a clean, enriched dataset containing only Falcon 9 launches.



Data Collection - Scraping

- Performed HTML GET requests to a static Wikipedia snapshot containing Falcon 9 launch records.
- Parsed the HTML content using BeautifulSoup.
- Identified the correct launch table among 25 tables on the page,.
- Extracted and cleaned column names (removing links, superscripts, annotations).
- Parsed each launch row to retrieve:
 - Flight number.
 - Customer.
 - Data and time.
 - Launch site.
 - Launch Outcome.
 - Booster Version.
 - Payload and payload mass.
 - Orbit
 - Booster landing result.
- Cleaned inconsistent formatting (Unicode and missing values)
- Built a structured Pandas Dataframe ready for merging with API data.



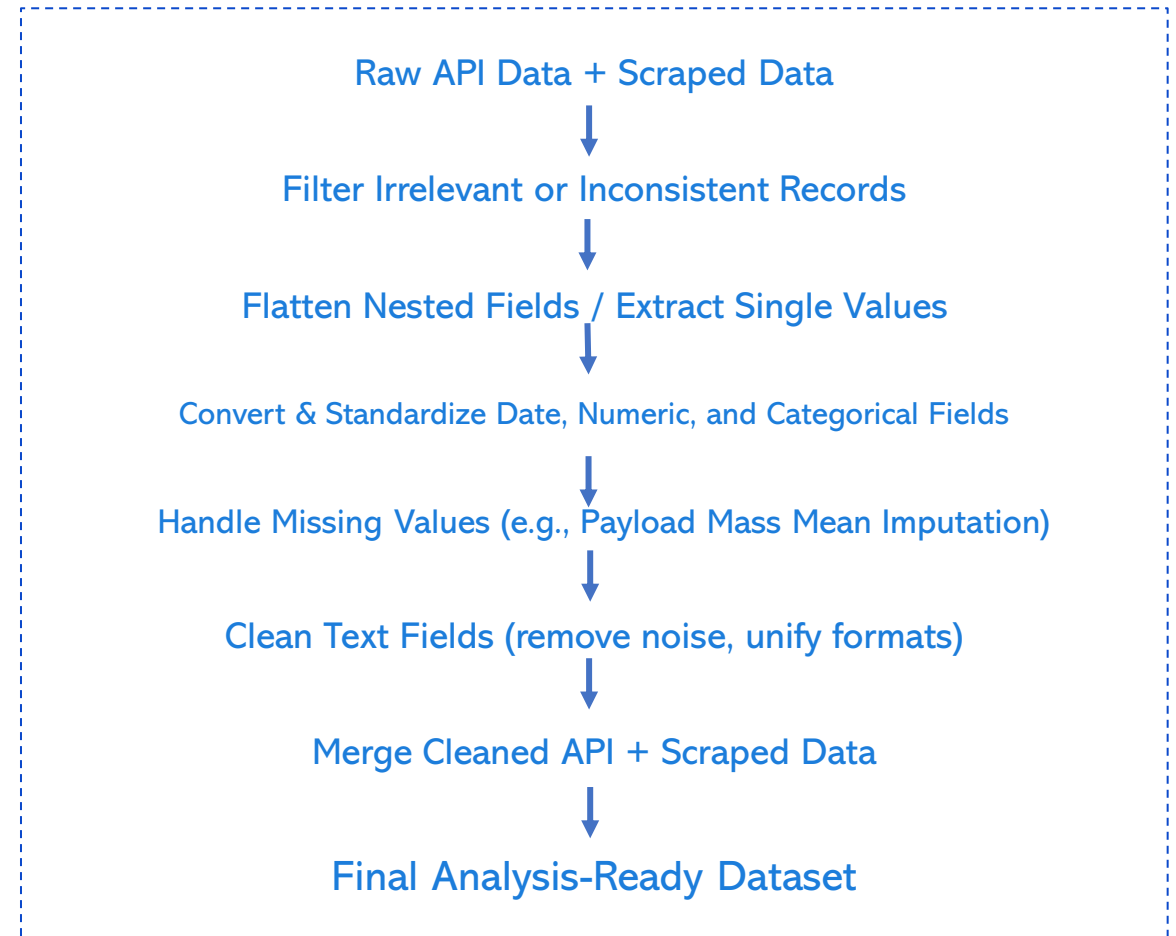
Data Wrangling

During the data wrangling phase, the raw data collected from the SpaceX REST API and the Wikipedia web scraping process was cleaned, standardized, and transformed into a consistent and analysis-ready dataset. This step ensured that all variables were properly formatted, missing values were handled, and the dataset contained only formatted, missing values were handled, and the dataset contained only relevant and reliable records for further EDA and predictive modeling.

Data Wrangling

- Filtered inconsistent records (e.g., launches with multiple cores or payloads).
- Extracted single-value fields from lists and nested structures.
- Converted date fields to proper datetime format and standardized them.
- Restricted launch dates to the project's required timeframe.
- Identified and handled missing values, replacing payload mass NaNs with the mean.
- Cleaned categorical fields (booster version, landing outcome, orbit, etc.).
- Merged enriched API data with scraped data into a unified structure.
- Ensured consistency across all features for modeling and visualization.

Github link: [labs-jupyter-spacex-Data wrangling.ipynb](#)



EDA with Data Visualization

1. Scatter Plots (Categorical and Continuous)

- **Flight Number vs. Payload Mass:** Used to observe if the success rate improved as SpaceX gained more experience (higher flight numbers) and how the mass of the payload affected the outcome.
- **Flight Number vs. Launch Site:** Plotted to identify site-specific trends. It shows which sites were used for early missions versus later ones and how success rates vary across locations like CCAFS, VAFB, and KSC.
- **Payload Mass vs. Launch Site:** This helped visualize the payload capacity limits of each launch site and if heavier payloads had a higher risk of failure at specific locations.
- **Flight Number vs. Orbit Type:** Used to see the progression of mission types over time and if certain orbits (like LEO or GTO) became more successful as the number of flights increased.
- **Payload Mass vs. Orbit Type:** Plotted to understand the relationship between the weight of the satellite and the target orbit, revealing that heavier payloads are often destined for Polar, LEO, or ISS orbits with high success rates.
- **Why?** These scatter plots (using `sns.catplot`) are essential for identifying correlations between multiple variables (like mass and site) while using color (`hue='Class'`) to instantly distinguish between successful and failed landings.

EDA with Data Visualization

2. Bar Chart (Success Rate by Orbit)

- **Orbit Type vs. Success Rate:** I calculated the mean success rate for each orbit type and represented it in a bar chart.
- **Why?** This chart is the best way to compare categorical performance. It clearly shows which orbits (e.g., SSO, HEO) have a 100% success rate versus those that are more challenging (e.g., GTO).

3. Line Chart (Yearly Success Trend)

- **Year vs. Success Rate:** After extracting the year from the date column, I plotted a line chart to show the evolution of SpaceX's reliability over time.
- **Why?** Line charts are ideal for temporal data. This visualization highlights the impressive upward trend in landing success from 2013 to 2020, showing the technological maturity of the Falcon 9 program.

Github link: [edataviz.ipynb](https://github.com/edataviz/edataviz.ipynb)

EDA with SQL

- **Launch Site Identification:** Retrieved the unique names of all launch sites in the SpaceX dataset using the DISTINCT operator.
- **Pattern-Based Filtering:** Selected the first five records for launch sites starting with the 'CCA' string to verify specific regional data.
- **Total Payload Calculation:** Calculated the total payload mass carried across all missions specifically commissioned by NASA (CRS).
- **Averaging Performance:** Determined the average payload mass for the 'F9 v1.1' booster version to understand its typical mission capacity.
- **Landing Milestones:** Identified the date of the first successful ground pad landing using the MIN function for temporal analysis.

EDA with SQL

- **Multicriteria Filtering:** Listed booster versions that achieved successful drone ship landings with payload masses between 4,000 kg and 6,000 kg.
- **Mission Outcome Summary:** Counted and categorized the total number of successful and failed mission outcomes.
- **Maximum Payload Analysis:** Utilized subqueries to identify all booster versions that have carried the maximum recorded payload mass.
- **Temporal Error Tracking:** Extracted specific drone ship landing failures from 2015, using string manipulation functions (substr) to filter by year and month.
- **Outcome Ranking:** Ranked the counts of landing outcomes in descending order within a specific date range (2010–2017) to measure operational reliability.

Github link: [jupyter-labs-eda-sql-coursera_sqlite.ipynb](https://github.com/jupyter-labs/eda-sql-coursera/blob/master/sqlite.ipynb)

Build an Interactive Map with Folium

Summary of Map Objects Created

- **Circles (folium.Circle):** Added centered on the coordinates of each launch site and the NASA Johnson Space Center.
 - Customized with specific radii and colors (e.g., orange or blue) to highlight specific areas.
- **Markers (folium.map.Marker):** Placed at the precise coordinates of every launch site.
 - **Marker Clusters:** Used to group multiple launch records that share the same coordinates, preventing map clutter.
 - **Custom Icons:** Implemented colored icons (green for success, red for failure) to represent launch outcomes.
- **Popups and Tooltips:** Attached to markers to display the launch site name and the specific outcome (Class 0 or 1) when clicked or hovered.
- **Mouse Position Tool:** An interactive plugin added to the top-right of the map to capture real-time latitude and longitude coordinates as the mouse moves.
- **PolyLines:** Used to draw lines between launch sites and points of interest like coastlines, railways, or highways.

Build an Interactive Map with Folium

Why These Objects Were Added

- **Spatial Context:** Plain coordinates are difficult to interpret. **Circles and Markers** provide an immediate visual understanding of where launch sites are geographically located relative to each other.
- **Performance Visualization:** By color-coding markers (**Marker Color**), we can visually identify "hotspots" of success or failure. For example, a cluster with many green markers indicates a highly reliable launch site.
- **Proximity Analysis:** Objects like **PolyLines** and the `calculate_distance` were implemented to analyze safety and logistics. For instance, determining how close a launch site is to the coastline or major infrastructure helps explain the strategic choice of site locations.
- **Interactivity:** The **Mouse Position** and **Popups** allow for exploratory data analysis, enabling the user to manually verify distances and details that aren't immediately obvious from a static data table.

Github link: [lab_jupyter_launch_site_location.ipynb](#)

Build a Dashboard with Plotly Dash

Summary of Dashboard Plots and Interactions

- **Launch Site Dropdown Menu:** A searchable dropdown component that allows users to select "All Sites" or specific locations like CCAFS LC-40, VAFB SLC-4E, KSC LC-39A, and CCAFS SLC-40.
- **Success Pie Chart:** A dynamic chart that displays the total successful launches by site when "All Sites" is selected, or the success vs. failure ratio for a specific site.
- **Payload Range Slider:** An interactive slider covering a range from 0 to 10,000 kg with increments of 1,000 kg, allowing users to filter data by cargo weight.
- **Success vs. Payload Scatter Chart:** A scatter plot that visualizes the correlation between Payload Mass and Launch Outcome (class), color-coded by Booster Version Category.

Build a Dashboard with Plotly Dash

Why These Plots and Interactions Were Added

- **Dynamic Filtering:** The **Dropdown Menu** was implemented to provide a customized view of the data, allowing users to drill down into the performance of individual launch sites rather than looking at aggregate numbers only.
- **Success Distribution:** The **Pie Chart** provides an immediate visual breakdown of reliability. It helps identify which sites contribute the most to SpaceX's overall success rate and, when filtered, reveals the specific failure-to-success ratio of a single location.
- **Technical Correlation:** The **Scatter Chart** was added to determine if heavier payloads negatively impact landing success. By including the **Booster Version** in the color scale, users can also see if specific rocket iterations (like Block 5) handle certain payload ranges more effectively than others.
- **Interactive Exploration:** The **Range Slider** combined with the scatter plot allows for real-time hypothesis testing. For example, a user can quickly see if there is a "danger zone" for landing success at specific weight intervals.

Github link: [Build an Interactive Dashboard with Plotly Dash.TXT](#)

Predictive Analysis (Classification)

Model Development Process

- The development followed a systematic approach, from data preparation to hyperparameter optimization, as visualized in the flowchart below:

1. Data Preparation and Pre-processing

- **Variable Assignment:** Created a NumPy array for the target variable Y from the 'Class' column and a feature set X from the remaining processed variables.
- **Standardization:** Applied *StandardScaler* to the feature set X to ensure all numeric variables were on the same scale, which is critical for algorithms like SVM and KNN.
- **Data Splitting:** Divided the dataset into training and testing sets using *train_test_split* with a test size of 20% and a *random_state* of 2 to ensure reproducible results.

Predictive Analysis (Classification)

2. Model Building and Hyperparameter Tuning

- **Algorithm Selection:** Built four distinct classification models:
 - **Logistic Regression:** Tuned for regularization (C) and solvers.
 - **Support Vector Machine (SVM):** Optimized for kernels (linear, rbf, poly, sigmoid) and gamma.
 - **Decision Tree:** Tuned for criteria, splitters, and maximum depth.
 - **K-Nearest Neighbors (KNN):** Optimized for the number of neighbors and algorithms.
- **Optimization:** Used *GridSearchCV* with 10-fold cross-validation for every model to systematically find the best combinations of hyperparameters.

Predictive Analysis (Classification)

3. Evaluation and Improvement

- **Accuracy Metrics:** Evaluated models using the `.score()` method on the test data to measure how well the models generalized to unseen launches.
- **Confusion Matrices:** Generated confusion matrices for each model to analyze specific errors, such as **False Positives** (predicting a success when it failed).
- **Success Comparison:** Most models (Logistic Regression, SVM, and KNN) achieved an identical validation accuracy of approximately **84.8%**.

Predictive Analysis (Classification)

4. Finding the Best Performing Model

- **Comparison:** After testing all algorithms, the **Decision Tree** model achieved the highest validation accuracy of **87.6%** during the tuning phase.
- **Generalization:** All models displayed similar performance on the small test set (**83.3%** accuracy), but the Decision Tree's slightly higher cross-validation score made it the top candidate for this specific dataset.

Github link: [SpaceX Machine Learning Prediction Part 5.ipynb](#)

Results

1. Exploratory Data Analysis (EDA) Results

The exploratory phase revealed critical patterns that influence the success of the first-stage recovery:

- **Overall Success Rate:** After processing the raw data, the average landing success rate was determined to be **66%**.
- **Launch Site Analysis:** CCAFS SLC 40 is the most active site with **55 launches**, followed by KSC LC 39A (22) and VAFB SLC 4E (13).
- **Orbit Influence:** Missions to orbits such as **ISS, VLEO, and PO** show a high frequency of success, whereas GTO orbits present a more varied distribution of outcomes.
- **Missing Data Handling:** The LandingPad variable showed a **28.8% null rate**, which directly correlates with missions where a pad landing was not attempted.
- **Payload Mass Normalization:** Missing values in PayloadMass were handled by calculating and imputing the **mean**, ensuring a complete dataset for further analysis.

Results

2. Interactive Analytics Demo

- The project utilizes several data engineering techniques to transform raw web data into actionable insights:
- **Dual-Source Data Collection:** The workflow demonstrates the integration of the **SpaceX API** for real-time technical data and **Web Scraping** (using BeautifulSoup) to capture historical records from Wikipedia.
- **Data Wrangling Pipeline:** Screenshots confirm the use of **Pandas and NumPy** to clean IDs, extract core data (like GridFins and ReusedCount), and handle the logic for separate launch events.
- **Structured Output:** The demo showcases the transition from unstructured HTML tables to a refined dataset_part_2.csv, featuring integrated technical variables and coordinates.

Results

3. Predictive Analysis Results

The foundations laid during the data preparation phase ensure the viability of the classification models:

- **Target Variable Engineering:** A binary **Class** column was created: **1** represents a successful landing (e.g., True ASDS, True RTLS) and **0** represents a failure (derived from a specific set of `bad_outcomes` like False Ocean or None None).
- **Feature Standardization:** Critical features such as `FlightNumber`, `PayloadMass`, and `Flights` (number of times the core has flown) were standardized to prevent bias in predictive algorithms.
- **Model Readiness:** The resulting dataset is balanced and formatted for supervised learning, allowing for the application of algorithms like Logistic Regression, SVM, and Decision Trees to predict landing success with high confidence.

Results

TASK 9

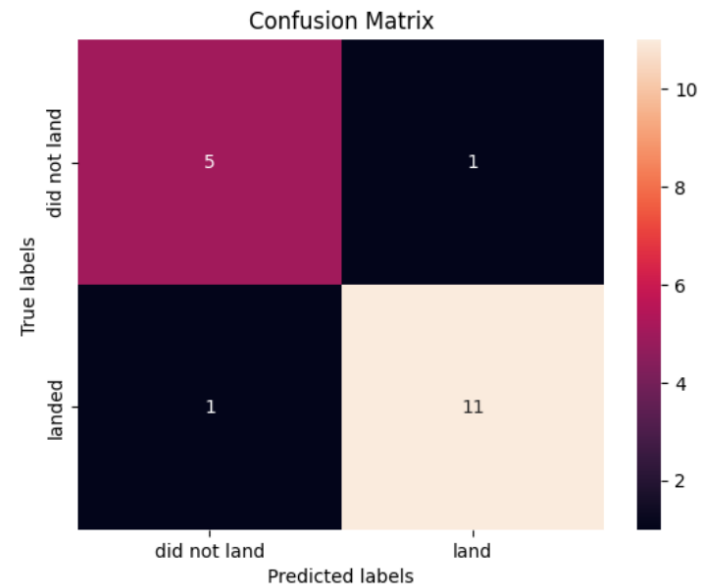
Calculate the accuracy of tree_cv on the test data using the method `score` :

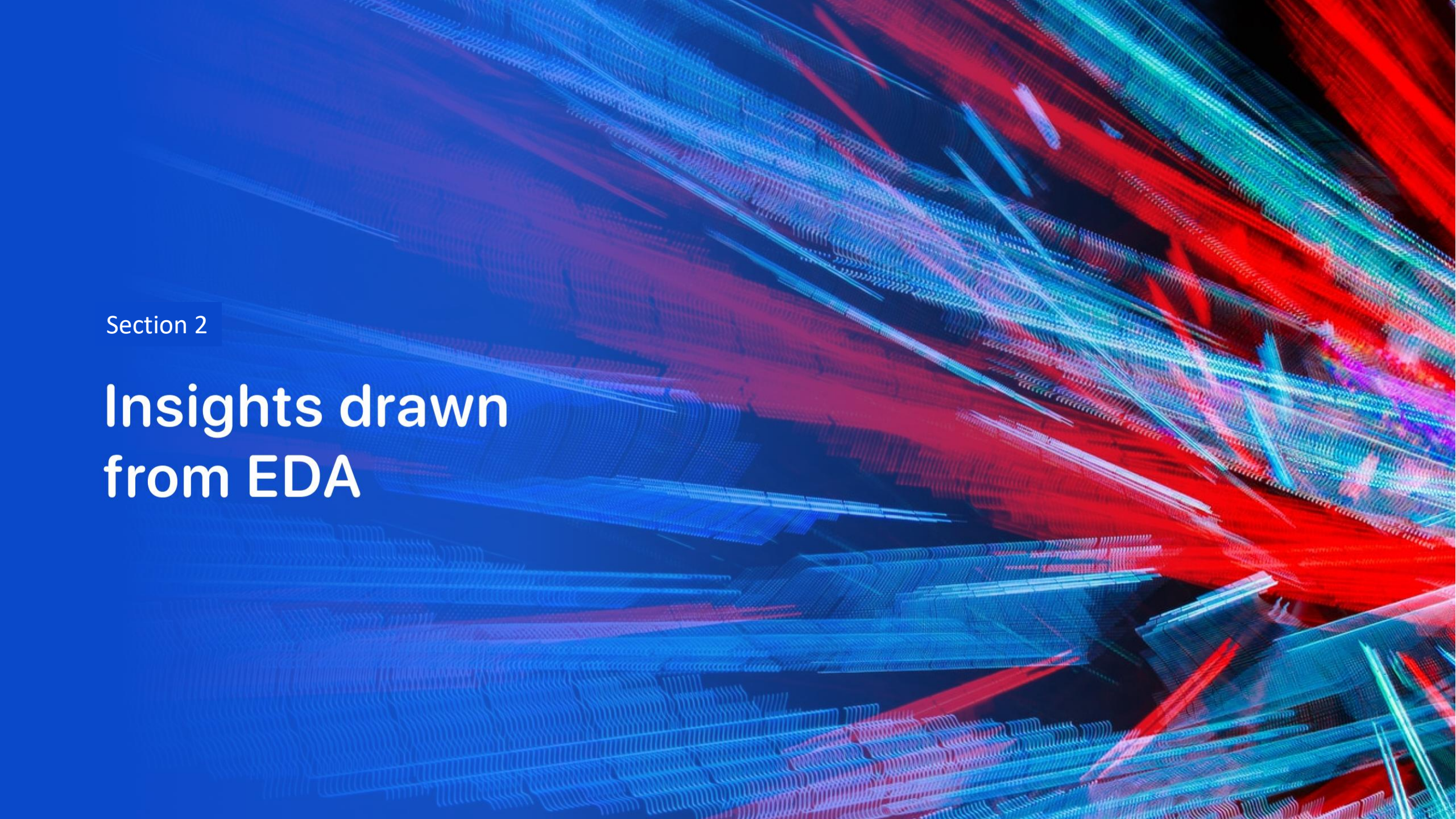
```
[52]: test_accuracy = tree_cv.score(X_test, Y_test)
      print("Test accuracy:", test_accuracy)
```

Test accuracy: 0.8888888888888888

We can plot the confusion matrix

```
[53]: yhat = tree_cv.predict(X_test)
      plot_confusion_matrix(Y_test, yhat)
```



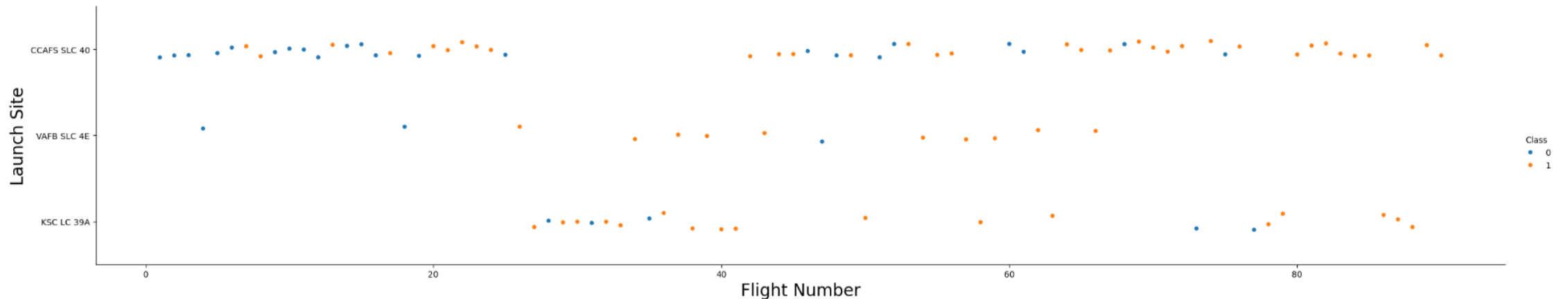
The background of the slide is an abstract composition. It features a dark blue gradient on the left side, which transitions into a complex pattern of diagonal streaks and lines in shades of blue, red, and teal on the right. These streaks have a textured, almost woven appearance, suggesting a digital or data-driven theme. The overall effect is dynamic and modern.

Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

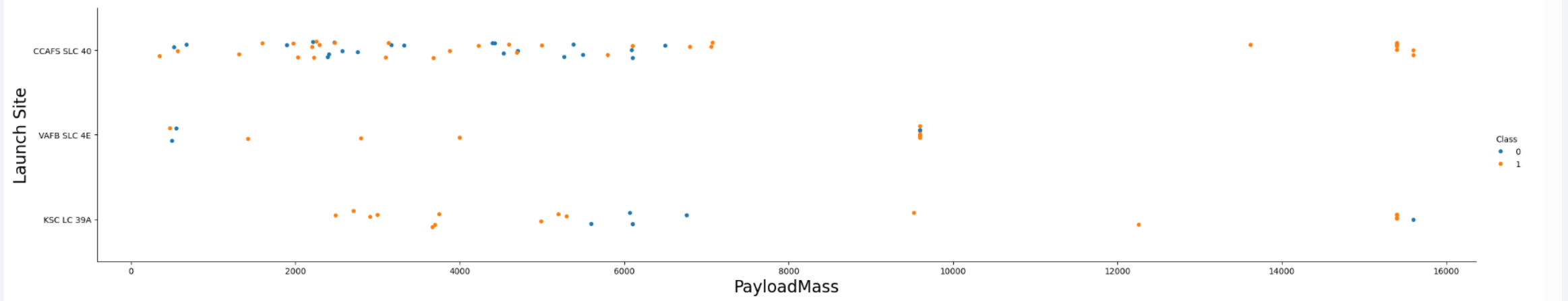
```
: sns.catplot( x="FlightNumber", y="LaunchSite", hue="Class", data=df, aspect=5)
plt.xlabel("Flight Number", fontsize=20)
plt.ylabel("Launch Site", fontsize=20)
plt.show()
```



The scatter plot shows that launch activity varies by site, with CCAFS dominating early missions and KSC appearing only in later ones. Landing success improves significantly with higher flight numbers across all sites, indicating technological progress rather than site-specific effects. Overall, Flight Number is the strongest predictor of landing success, not the launch site.

Payload vs. Launch Site

```
sns.catplot( x="PayloadMass", y="LaunchSite", hue="Class", data=df, aspect=5)  
plt.xlabel("PayloadMass", fontsize=20)  
plt.ylabel("Launch Site", fontsize=20)  
plt.show()
```



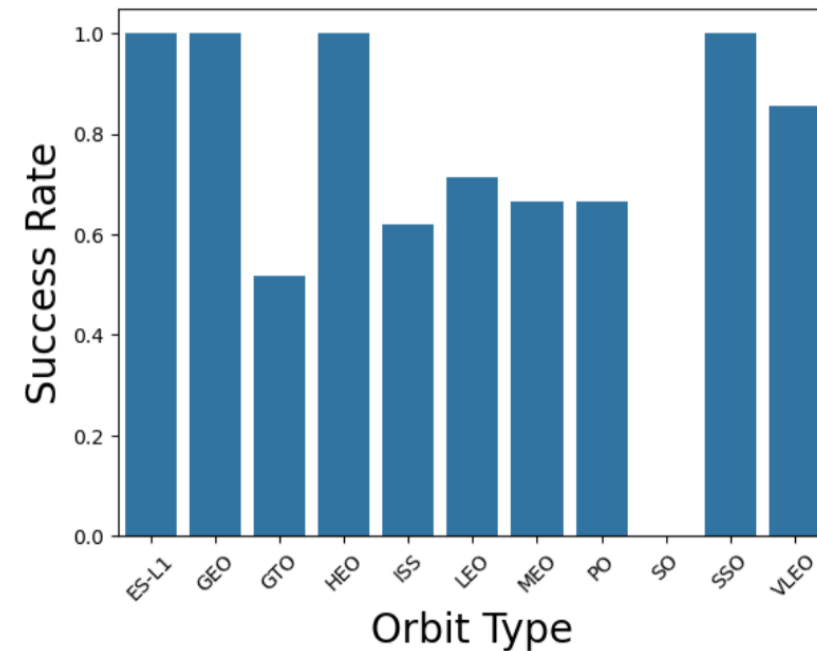
This scatter plot illustrates that landing success (orange) increases significantly with payloads over 10,000 kg, particularly at KSC LC 39A and VAFB SLC 4E. While CCAFS SLC 40 handles most light missions with mixed results, heavy payloads show a consistent success rate across all launch sites.

Success Rate vs. Orbit Type

Orbit type does influence success rate, but mostly because different orbits correspond to missions of different difficulty levels and different points in SpaceX's technological evolution. Higher-energy orbits (like GTO) show more failures, while later-stage orbits (like SSO or GEO) show high success due to improved landing technology

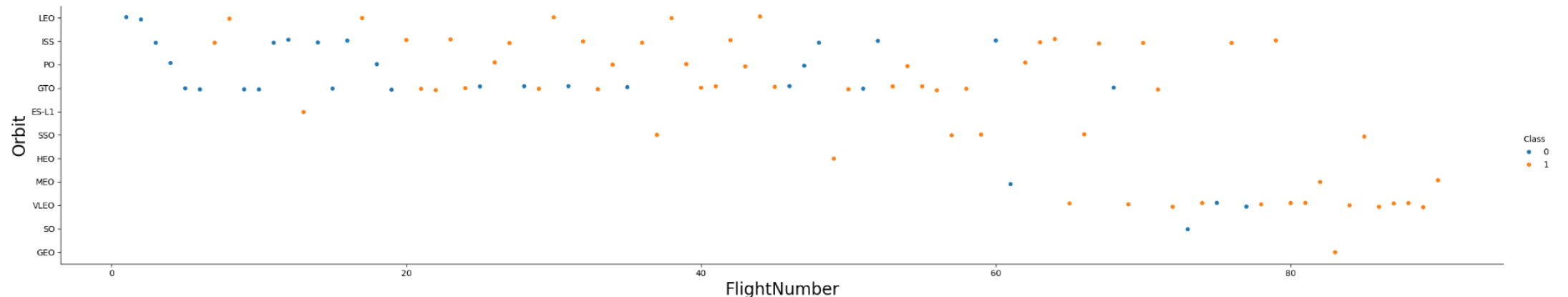
```
[6]: # Compute success rate per orbit type
orbit_success = df.groupby("Orbit")["Class"].mean().reset_index()

# Plot bar chart
sns.barplot(x="Orbit", y="Class", data=orbit_success)
plt.xticks(rotation=45)
plt.xlabel("Orbit Type", fontsize=20)
plt.ylabel("Success Rate", fontsize=20)
plt.show()
```



Flight Number vs. Orbit Type

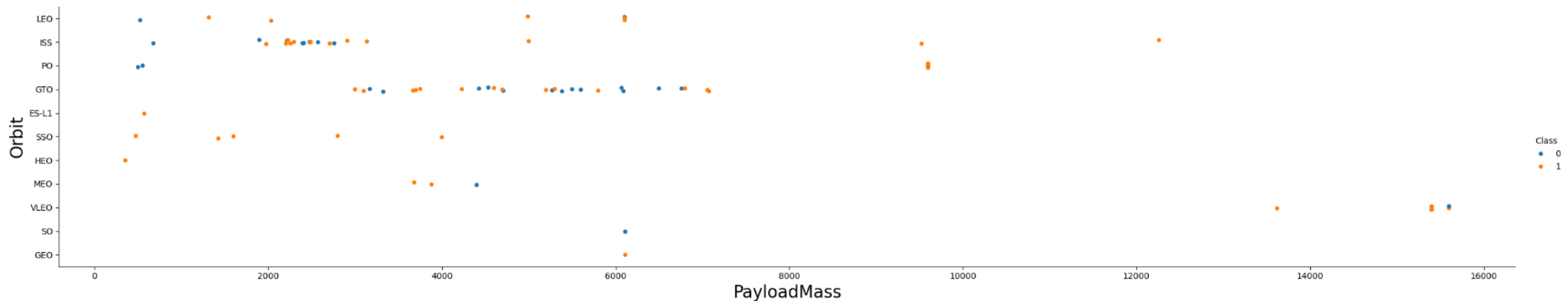
```
: # Plot a scatter point chart with x axis to be FlightNumber and y axis to be the Orbit, and hue to be the class value
sns.catplot( x="FlightNumber", y="Orbit", hue="Class", data=df, aspect=5)
plt.xlabel("FlightNumber", fontsize=20)
plt.ylabel("Orbit", fontsize=20)
plt.show()
```



- Show a scatter point of Flight number vs. Orbit type
- The LEO orbit, success seems to be related to the number of flights. Conversely, in the GTO orbit, there appears to be no relationship between flight number and success.

Payload vs. Orbit Type

```
[8]: sns.catplot( x="PayloadMass", y="Orbit", hue="Class", data=df, aspect=5)
plt.xlabel("PayloadMass", fontsize=20)
plt.ylabel("Orbit", fontsize=20)
plt.show()
```



- With heavy payloads the successful landing or positive landing rate are more for Polar,LEO and ISS.
- However, for GTO, it's difficult to distinguish between successful and unsuccessful landings as both outcomes are present.

Launch Success Yearly Trend

The success rate since 2013 kept increasing till 2020.



All Launch Site Names

- **Query Result:** The query returns a list of unique names for the launch sites found in the SPACEXTABLE. The output displays four entries: CCAFS LC-40, VAFB SLC-4E, KSC LC-39A, and CCAFS SLC-40.
- **Query Explanation**
 - The DISTINCT clause is used on the "Launch_Site" column to filter out duplicate values. This allows you to identify the specific geographical locations used by SpaceX within the my_data1.db database.

```
[11]: %%sql SELECT *  
      DISTINCT "Launch_Site"  
      FROM SPACEXTABLE;
```

```
* sqlite:///my_data1.db  
Done.
```

```
[11]: Launch_Site  
-----  
      CCAFS LC-40  
      VAFB SLC-4E  
      KSC LC-39A  
      CCAFS SLC-40
```


Launch Site Names Begin with 'CCA'

```
[18]: %%sql SELECT *  
FROM SPACEXTABLE  
WHERE "Launch_Site" LIKE 'CCA%'  
LIMIT 5;
```

```
* sqlite:///my_data1.db  
Done.
```

```
[18]:
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

- Query Result: The query returns the first 5 records from the dataset where the launch site starts with the characters "CCA". The resulting table displays detailed launch data, including the date, time, booster version, and outcomes for five specific missions launched from CCAFS LC-40 between 2010 and 2013.
- Query explanation:
 - SELECT *: Instructs the database to retrieve all columns for the matching rows.
 - WHERE "Launch_Site" LIKE 'CCA%': Uses a pattern matching filter to find only those entries where the launch site name begins with "CCA" (the % acts as a wildcard for any subsequent characters).
 - LIMIT 5: Restricts the output to only the first five rows found that meet the criteria, making the data easier to preview.

Total Payload Mass

```
[21]: %%sql SELECT SUM(PAYLOAD_MASS__KG_) AS Total_Payload_Mass
      FROM SPACEXTABLE
      WHERE Customer = 'NASA (CRS)';

* sqlite:///my_data1.db
Done.

[21]: Total_Payload_Mass
      45596
```

- Query Result: The output is a single value, 45596, labeled under the column alias Total_Payload_Mass.
- Query explanation:
 - SUM(PAYLOAD_MASS__KG_): This aggregate function calculates the total weight by adding up all values in the payload mass column for the filtered rows.
 - AS Total_Payload_Mass: This renames the resulting column for better readability in the final output.
 - WHERE Customer = 'NASA (CRS)': This filter ensures that only missions conducted for the specific customer "NASA (CRS)" are included in the calculation.

Average Payload Mass by F9 v1.1

- **Query Result:** The query returns a single numerical value: 2928.4. This result is displayed under the column header Avg_Payload_Mass and represents the calculated average weight (in kilograms) of all payloads launched using a specific rocket version.
- **Query Explanation**
 - `SELECT AVG(PAYLOAD_MASS__KG_)`: Uses the AVG aggregate function to calculate the mean value of the payload mass column.
 - `AS Avg_Payload_Mass`: Assigns a readable alias to the resulting column.
 - `FROM SPACEXTABLE`: Specifies the source table containing the SpaceX launch data.
 - `WHERE Booster_Version = 'F9 v1.1'`: Applies a filter so that only records involving the booster version F9 v1.1 are included in the average calculation.

Task 4

Display average payload mass carried by booster version F9 v1.1

```
[22]: %%sql SELECT AVG(PAYLOAD_MASS__KG_) AS Avg_Payload_Mass
      FROM SPACEXTABLE
      WHERE Booster_Version = 'F9 v1.1';
```

```
* sqlite:///my_data1.db
```

Done.

```
[22]: Avg_Payload_Mass
```

```
2928.4
```

First Successful Ground Landing Date

- **Query Result:** The query returns a single date: 2015-12-22, this result is displayed under the column alias First_Ground_Pad_Success, and It identifies the exact date when the very first successful landing on a ground pad was achieved by SpaceX.
- **Query Explanation**
 - **SELECT MIN(Date):** Uses the MIN aggregate function to find the earliest (minimum) value in the Date column.
 - **AS First_Ground_Pad_Success:** Assigns a descriptive alias to the output column for better clarity.
 - **FROM SPACEXTABLE:** Specifies the database table where the mission records are stored.
 - **WHERE Landing_Outcome = 'Success (ground pad)':** Filters the data to only consider missions where the landing was officially recorded as a success on a ground pad.

Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint: Use min function

```
[24]: %%sql SELECT MIN(Date) AS First_Ground_Pad_Success
      FROM SPACEXTABLE
      WHERE Landing_Outcome = 'Success (ground pad)';
```

```
* sqlite:///my_data1.db
Done.
```

```
[24]: First_Ground_Pad_Success
      2015-12-22
```

Successful Drone Ship Landing with Payload between 4000 and 6000

- **Query result:** The output displays a list of four specific booster versions—F9 FT B1022, F9 FT B1026, F9 FT B1021.2, and F9 FT B1031.2—that met all the required criteria defined in the mission task.
- **Query explanation:**
 - **SELECT (Booster_Version):** This command retrieves the names of the boosters from the specified column.
 - **FROM SPACEXTABLE:** This identifies the source table containing the SpaceX dataset.
 - **WHERE Landing_Outcome = 'Success (drone ship)':** This filter restricts the results to missions where the first stage successfully landed on a drone ship.
 - **AND PAYLOAD_MASS__KG_ > 4000:** This condition ensures only missions with a payload mass exceeding 4,000 kg are included.
 - **AND PAYLOAD_MASS__KG_ < 6000:** This final condition further limits the results to missions with a payload mass of less than 6,000 kg.

```
%%sql SELECT (Booster_Version)
FROM SPACEXTABLE
WHERE Landing_Outcome = 'Success (drone ship)'
      AND PAYLOAD_MASS__KG_ > 4000
      AND PAYLOAD_MASS__KG_ < 6000;
```

```
* sqlite:///my_data1.db
Done.
```

```
Booster_Version
```

```
F9 FT B1022
```

```
F9 FT B1026
```

```
F9 FT B1021.2
```

```
F9 FT B1031.2
```

Total Number of Successful and Failure Mission Outcomes

- **Query result:** The output displays a summary table categorized by the "Mission_Outcome" column, showing that there was 1 "Failure (in flight)", 98 standard "Success" cases, an additional single "Success" entry, and 1 case where the "Success (payload status unclear)".
- **Query explanation:**
 - **SELECT Mission_Outcome, COUNT(*) AS Total:** This part of the query selects the mission result category and counts the total number of occurrences for each, renaming the count column as "Total".
 - **FROM SPACEXTABLE:** This specifies that the data is being retrieved from the main SpaceX dataset table.
 - **GROUP BY Mission_Outcome:** This clause aggregates the data into distinct groups based on the different types of mission outcomes recorded in the database.

```
[17]: %%sql
      SELECT Mission_Outcome, COUNT(*) AS Total
      FROM SPACEXTABLE
      GROUP BY Mission_Outcome;
```

```
* sqlite:///my_data1.db
Done.
```

```
[17]:
```

Mission_Outcome	Total
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Boosters Carried Maximum Payload

Query result: The output consists of a list of twelve booster versions, specifically various Falcon 9 Block 5 (F9 B5) models, that are associated with the maximum payload mass recorded in the dataset.

Query explanation:

- **SELECT Booster_Version:** This part of the statement identifies the specific column to be retrieved from the database.
- **FROM SPACEXTABLE:** This defines the source table where the flight records are stored.
- **WHERE PAYLOAD_MASS__KG_ = (...):** This filter ensures that only the boosters that carried the highest possible weight are selected.
- **Subquery (SELECT MAX...):** A nested query is used to first calculate the single highest value present in the payload mass column across the entire table.

```
[18]: %%sql
      SELECT Booster_Version
      FROM SPACEXTABLE
      WHERE PAYLOAD_MASS__KG_ = (
          SELECT MAX(PAYLOAD_MASS__KG_)
          FROM SPACEXTABLE
      );

* sqlite:///my_data1.db
Done.

[18]: Booster_Version
      F9 B5 B1048.4
      F9 B5 B1049.4
      F9 B5 B1051.3
      F9 B5 B1056.4
      F9 B5 B1048.5
      F9 B5 B1051.4
      F9 B5 B1049.5
      F9 B5 B1060.2
      F9 B5 B1058.3
      F9 B5 B1051.6
      F9 B5 B1060.3
      F9 B5 B1049.7
```

2015 Launch Records

Query result: The output consists of a table listing two specific missions from the year 2015 that resulted in a "Failure (drone ship)" landing outcome, identifying that these incidents occurred in January (01) and April (04) using booster versions F9 v1.1 B1012 and F9 v1.1 B1015, both launched from the CCAFS LC-40 site.

Query explanation:

- `SELECT substr(Date, 6, 2) AS Month`: This uses the substring function to extract the characters representing the month from the date string and renames the column as "Month".
- `Landing_Outcome, Booster_Version, Launch_Site`: These specific columns are selected to provide context for each failed landing attempt.
- `FROM SPACEXTABLE`: This specifies the table containing the SpaceX mission data as the source for the query.
- `WHERE substr(Date, 0, 5) = '2015'`: This filter uses a substring function to isolate the year from the date string, ensuring only records from 2015 are considered.
- `AND Landing_Outcome LIKE 'Failure (drone ship)%'`: This condition further filters the results to only include missions where the landing outcome specifically involved a failure on a drone ship.

```
[20]: %%sql
SELECT
    substr(Date, 6, 2) AS Month,
    Landing_Outcome,
    Booster_Version,
    Launch_Site
FROM SPACEXTABLE
WHERE substr(Date, 0, 5) = '2015'
    AND Landing_Outcome LIKE 'Failure (drone ship)%';

* sqlite:///my_data1.db
Done.
```

```
[20]:
```

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

Query result: The output displays a ranked list of landing outcomes and their respective counts for missions conducted between June 4, 2010, and March 20, 2017, showing that "No attempt" was the most frequent result with 10 occurrences, followed by 5 successful drone ship landings and 5 drone ship failures, continuing down to a single instance of a mission being "Precluded (drone ship)".

Query explanation:

- **SELECT Landing_Outcome, COUNT(*) AS Total:** This selects the landing result category and calculates the total number of times each outcome occurred, labeling the count as "Total".
- **FROM SPACEXTABLE:** This identifies the source table containing the SpaceX mission data.
- **WHERE Date BETWEEN '2010-06-04' AND '2017-03-20':** This clause restricts the analysis to a specific historical timeframe.
- **GROUP BY Landing_Outcome:** This aggregates the individual mission records into groups based on their specific landing results.
- **ORDER BY Total DESC:** This sorts the final list in descending order, placing the most common landing outcomes at the top of the table.

```
[21]: %%sql
SELECT
    Landing_Outcome,
    COUNT(*) AS Total
FROM SPACEXTABLE
WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'
GROUP BY Landing_Outcome
ORDER BY Total DESC;
```

* sqlite:///my_data1.db
Done.

```
[21]:
```

Landing_Outcome	Total
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

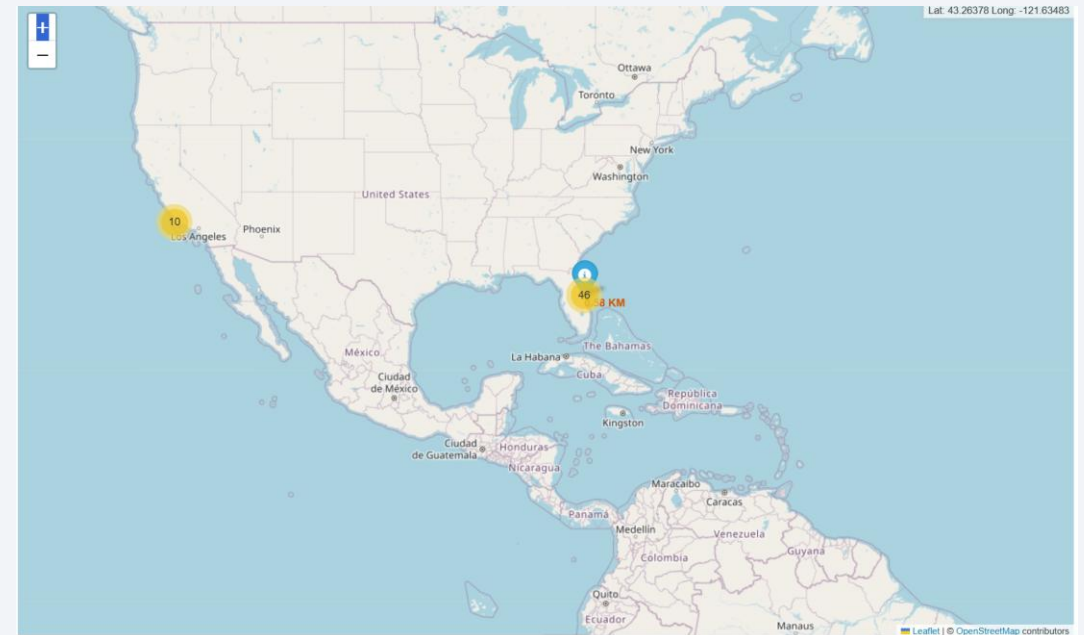
A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The background is a deep blue gradient.

Section 3

Launch Sites Proximities Analysis

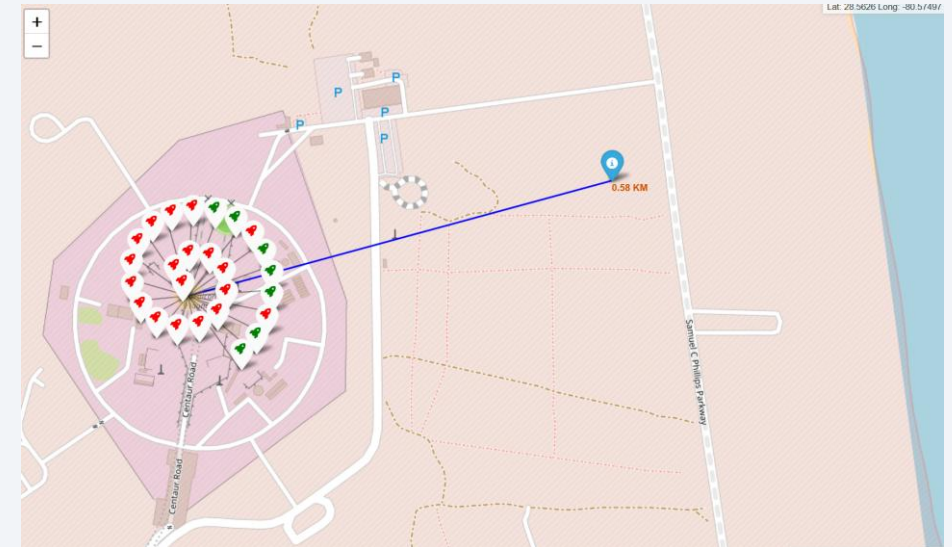
Global map of launch sites from SpaceX

- The Folium map reveals SpaceX's strategic launch sites on the East and West coasts of the U.S., highlighting a clear preference for coastal areas. Key elements include Marker Clusters that organize high mission volumes in Florida (46 launches) and California (10 launches), along with distance tags confirming immediate proximity to the coastline. Interactive features, such as the mouse position tracker, allow for precise coordinate verification of each site.
- These findings confirm that site selection follows strict safety protocols, ensuring launch trajectories and landing attempts occur over the ocean to protect populated areas. The analysis identifies Florida as SpaceX's primary operational hub due to its higher launch density. This geospatial approach utilizes the Haversine formula to validate that sites are both safe and strategically positioned near critical transport infrastructure.



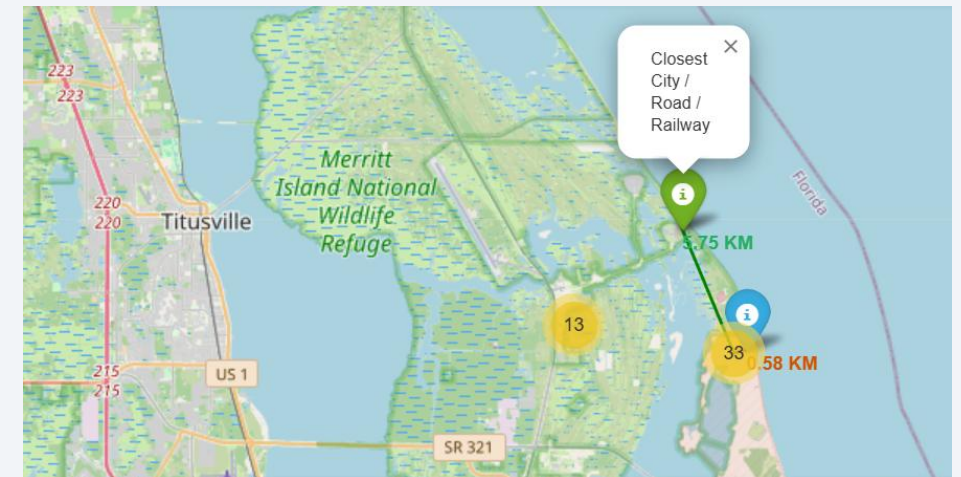
Geospatial Analysis of SpaceX Launch Outcomes and Proximity to Infrastructure

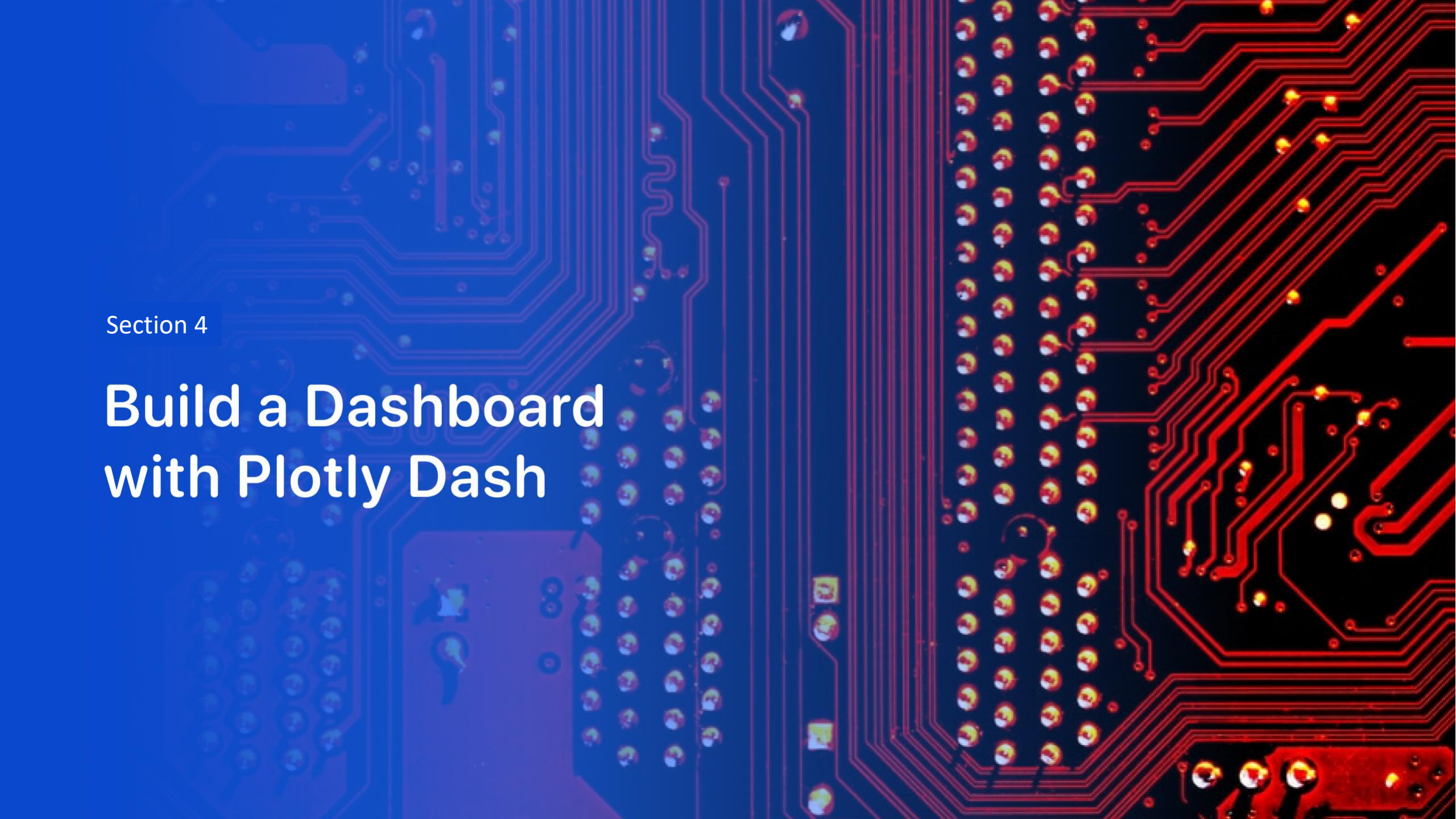
- This interactive Folium map visualizes the precise coordinates of launch records at a specific site, utilizing color-coded markers to represent mission outcomes. Green markers indicate successful landings, while red markers represent failed attempts, allowing for an immediate visual assessment of site-specific reliability. The map also features a PolyLine and a distance tag, which calculates the proximity between the launch complex and nearby geographical points, such as coastlines or infrastructure, using the Haversine formula.
- The findings in this screenshot highlight a high density of operations within a localized area, where success and failure rates can be analyzed geographically. A key observation is the strategic distance of 0.58 KM from the launch pad to the nearest point of interest, illustrating SpaceX's focus on minimizing risks to surrounding infrastructure. By clustering these color-labeled outcomes, the map provides a data-driven justification for evaluating how different launch sites and environmental factors correlate with the overall landing success of the Falcon 9 booster.



Proximity Analysis of SpaceX Launch Site to Critical Infrastructure and Coastline

- This Folium map screenshot illustrates the proximity analysis of a SpaceX launch site in Florida to critical surrounding infrastructure. Key elements include the PolyLines and numerical distance markers, such as 5.75 KM and 0.58 KM, which calculate the shortest path between the launch complex and logistical points like the nearest railway, highway, and coastline. The use of Marker Clusters (labeled 13 and 33) effectively groups high-density mission data, while custom pop-up labels identify specific points of interest for safety and operational assessment.
- The findings reveal a clear strategic pattern: launch sites are positioned extremely close to the coastline (0.58 KM) to facilitate safe over-water flight paths and sea-based recoveries. Additionally, the proximity to major transport infrastructure, such as the railway and US 1 highway, ensures efficient logistics for transporting heavy rocket components. This geospatial analysis demonstrates that site selection balances stringent safety requirements for minimizing inland risk with the practical necessity of being near large-scale transportation networks.

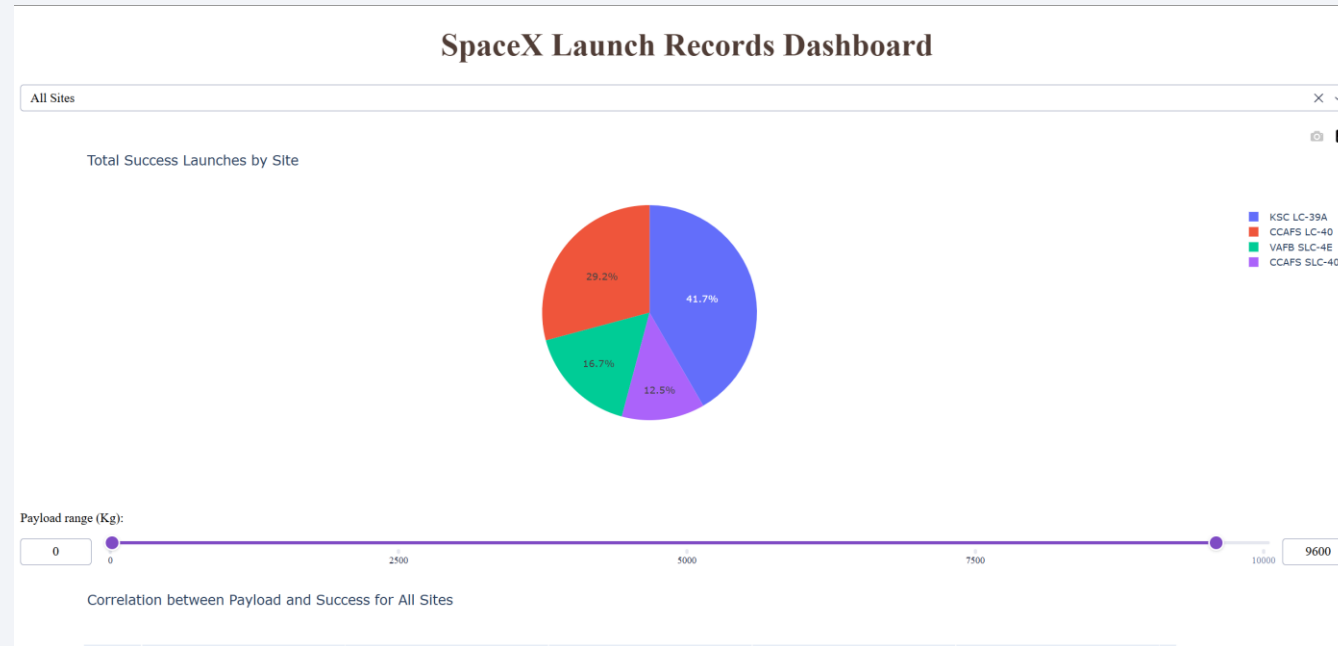




Section 4

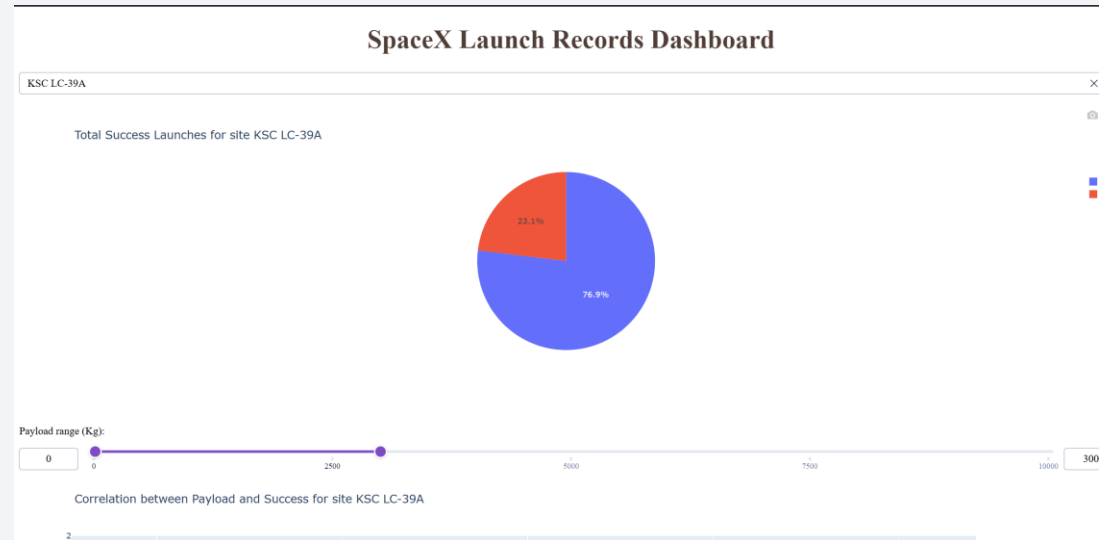
Build a Dashboard with Plotly Dash

SpaceX Total Success Launches by site



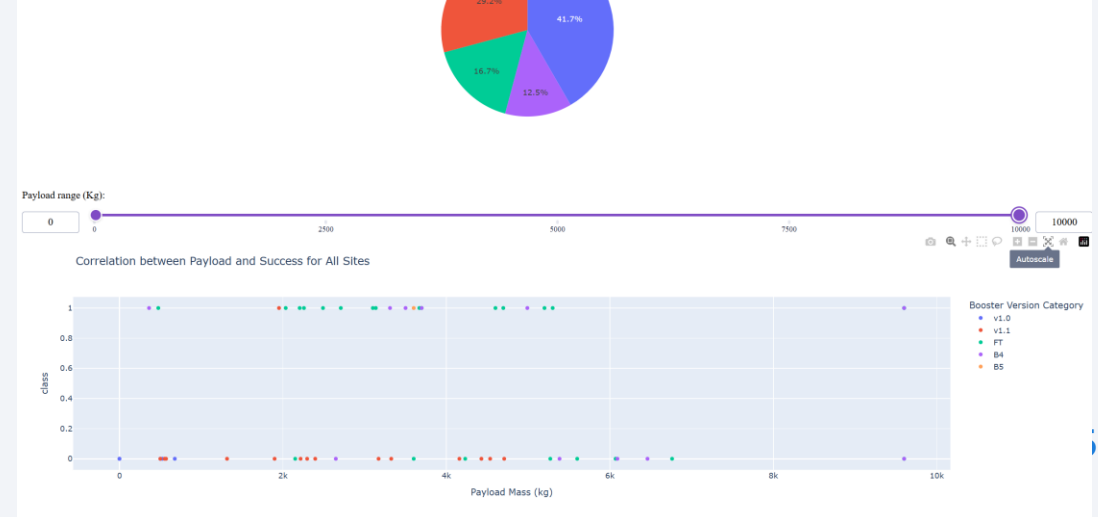
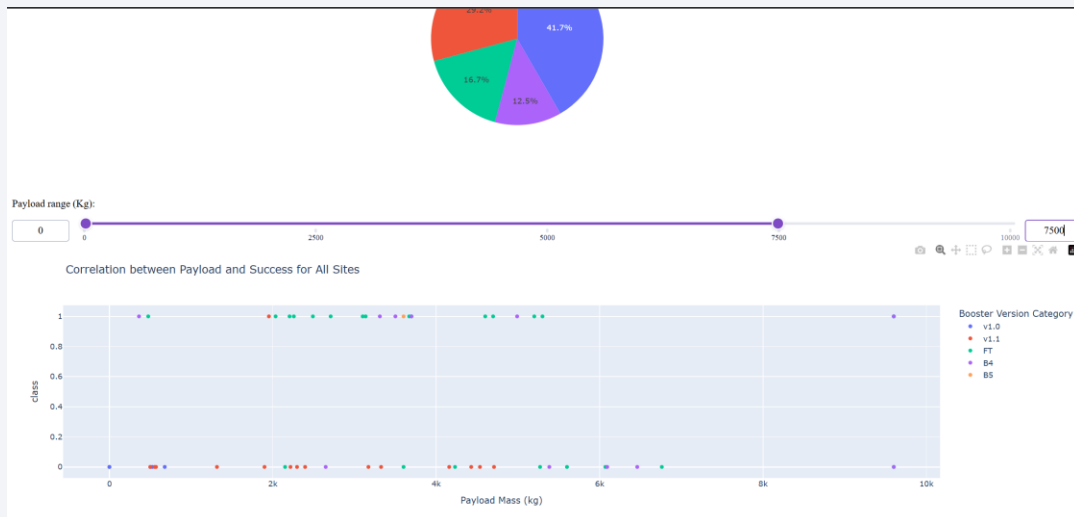
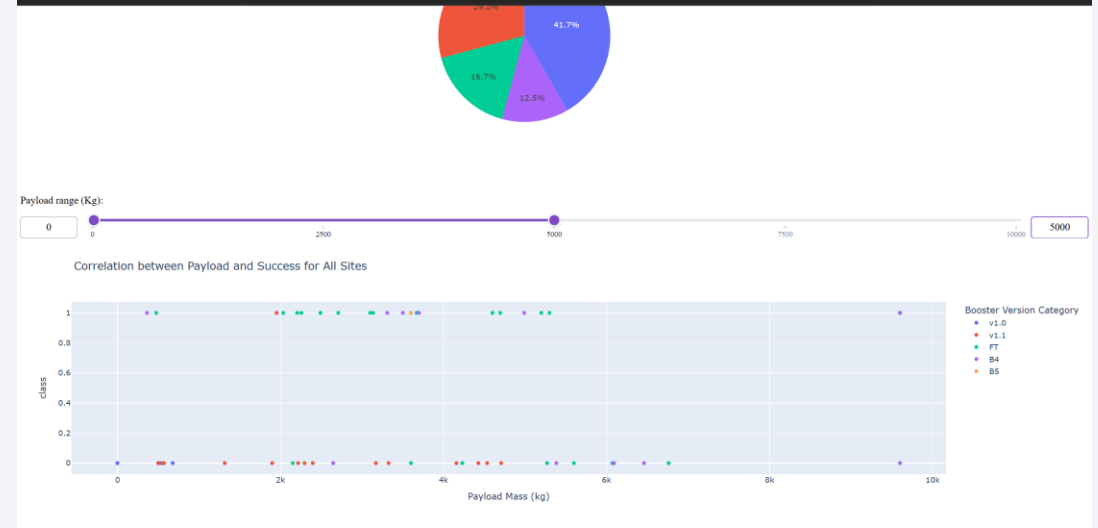
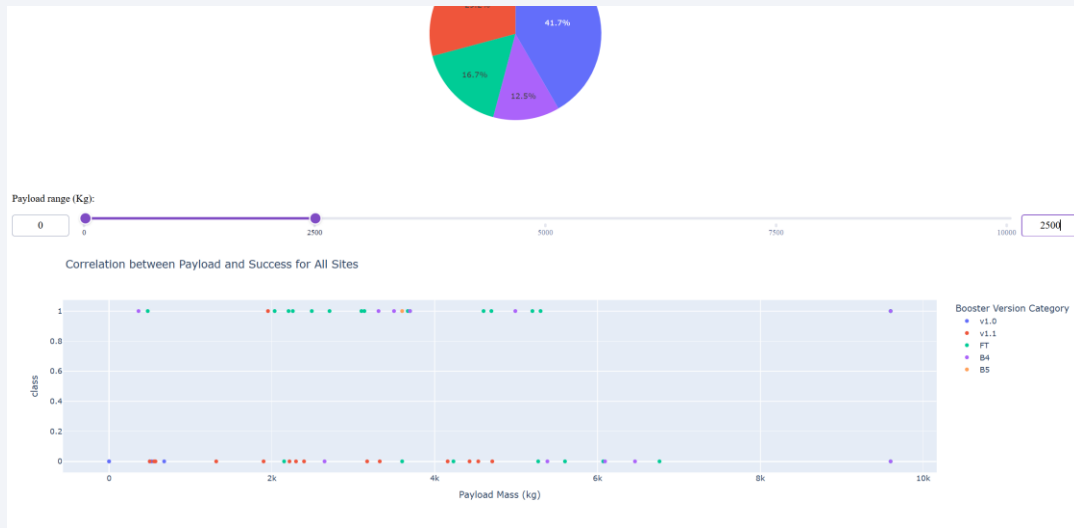
- This dashboard features three interactive components to analyze mission reliability : a Site Selection Dropdown to filter results by location, a Success Pie Chart showing that KSC LC-39A holds the largest share of successful launches (41.7%), and a Payload Range Slider to isolate specific cargo weights. The data confirms that KSC LC-39A and CCAFS LC-40 are the most successful hubs, together accounting for over 70% of total successful missions. By correlating payload mass with landing outcomes via the scatter plot, the dashboard identifies how weight and booster versions influence recovery success, providing the foundational insights needed for predictive modeling.

Performance Analysis and Most Success Rate of SpaceX Launch Site: KSC LC-39A



- This dashboard screenshot highlights the performance of the KSC LC-39A launch site, featuring a site-specific Success Pie Chart and a Payload Range Slider set between 0 and 3,000 kg. The primary finding is the site's high operational reliability, with a 76.9% success rate (represented by the blue "1" segment) compared to a 23.1% failure rate. By filtering for lower payload masses, the dashboard allows users to observe the correlation between lighter cargo missions and landing success for this specific location, confirming that KSC LC-39A is a highly effective hub for SpaceX's recovery operations within this weight category.

Analysis of Payload and Booster Performance on Launch Success



Analysis of Payload and Booster Performance on Launch Success

- This dashboard view provides a detailed correlation analysis between payload mass and mission success across all SpaceX launch sites. A key finding is that the FT (Full Thrust) and B5 (Block 5) booster versions demonstrate the highest success rates, appearing consistently at the top of the "class 1" success axis across various weight categories. In terms of payload range, missions between 2,000 kg and 4,000 kg show a significant concentration of successful landings, while higher mass missions (above 8,000 kg) also maintain a strong success profile, particularly with the B5 booster. Conversely, early versions like v1.1 are primarily clustered in the failure category (class 0) at lower payload masses, illustrating the substantial technological reliability gains achieved through booster iterations and optimized cargo handling.

Section 5

Predictive Analysis (Classification)

Classification Accuracy

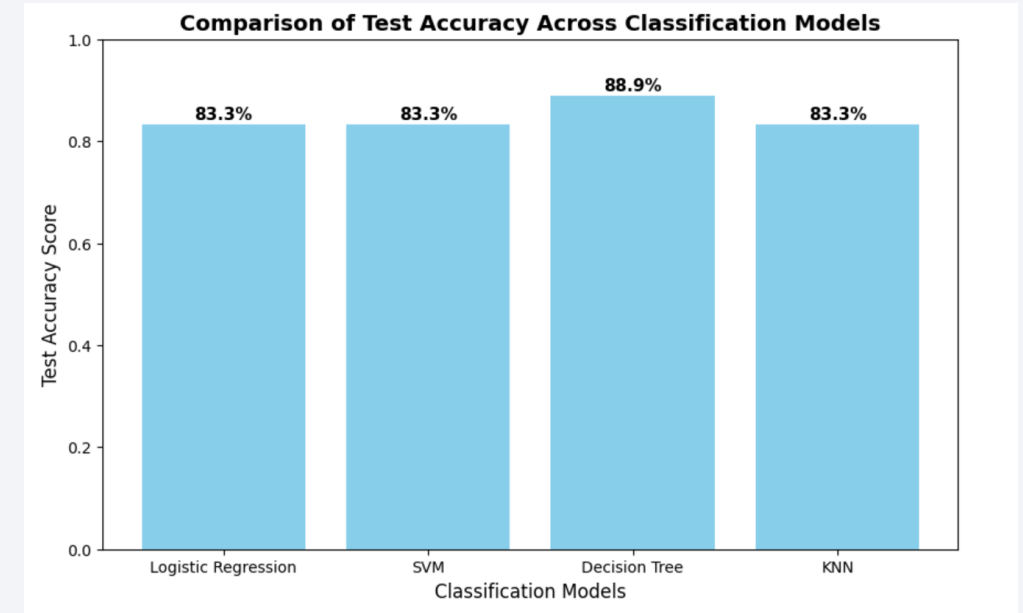
Model Comparison and Selection

To find the model with the highest classification accuracy, we evaluated four different algorithms using the same standardized dataset. After performing 10-fold cross-validation with GridSearchCV and testing each model on an unseen test set (20% of the data), the results were summarized as follows:

- **Logistic Regression:** Achieved a test accuracy of approximately 0.833 (83.3%).
- **Support Vector Machine (SVM):** Achieved a test accuracy of approximately 0.833 (83.3%).
- **K-Nearest Neighbors (KNN):** Achieved a test accuracy of approximately 0.833 (83.3%).
- **Decision Tree Classifier:** Outperformed the other models with a test accuracy of approximately 0.889 (88.9%).
- **The Winning Model:** Decision Tree

The Decision Tree was identified as the best-performing model for this classification task. It achieved the highest predictive accuracy by correctly identifying both successful landings and failed landings more reliably than the other algorithms.

The slightly higher validation score (87.6%) during the hyperparameter tuning phase also suggested that the Decision Tree was better at capturing the specific non-linear relationships and patterns within the SpaceX mission features, such as payload mass and orbit types.



Confusion Matrix

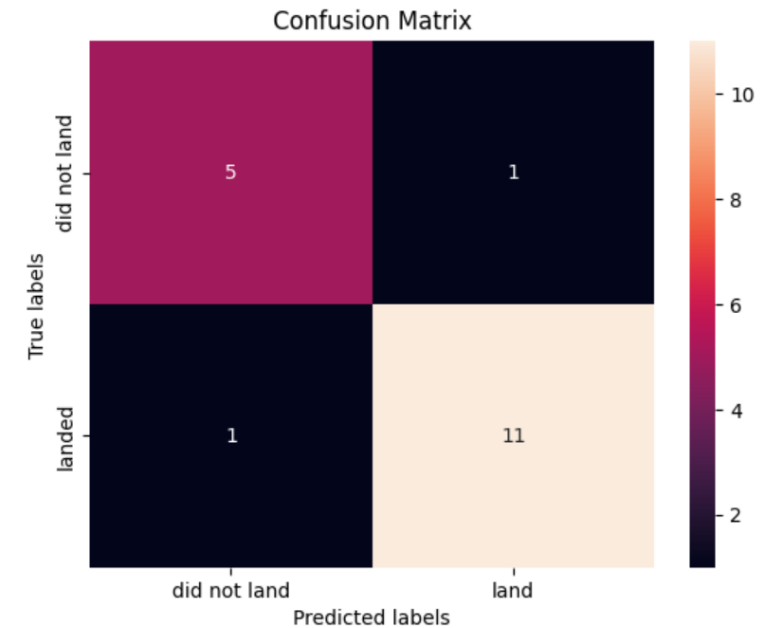
The Decision Tree model achieved a high predictive accuracy of 88.9% on the test data, correctly identifying the landing outcome for 16 out of 18 samples. Specifically, the confusion matrix shows that the model successfully predicted 11 landings (True Positives) and 5 failed landings (True Negatives). However, it recorded two errors: one false positive, where a failed landing was incorrectly predicted as successful, and one false negative, where a successful landing was classified as a failure. Despite these minor discrepancies, the model demonstrates strong reliability in forecasting Falcon 9 landing outcomes based on the provided mission features.

```
[52]: test_accuracy = tree_cv.score(X_test, Y_test)
      print("Test accuracy:", test_accuracy)
```

Test accuracy: 0.8888888888888888

We can plot the confusion matrix

```
[53]: yhat = tree_cv.predict(X_test)
      plot_confusion_matrix(Y_test, yhat)
```



Conclusions

- **Technological Evolution and Maturity:** The historical data shows a clear upward trend in landing success rates over time. Since the first successful landings in 2015-2016, SpaceX's reliability has increased significantly, demonstrating a steep and successful learning curve in reusable rocket technology.
- **Strategic Site Selection:** Geospatial analysis via Folium confirms that all launch sites are strategically located near coastlines. This is a critical safety measure to ensure that any debris or failed landing attempts fall into the ocean rather than populated areas, while also being close to major transport infrastructure (highways and railways) for logistics.
- **Orbit and Payload Sensitivity:** EDA and interactive dashboard analysis reveal that success rates are highly dependent on mission profiles. For instance, missions to Low Earth Orbit (LEO) generally show higher landing success rates compared to Geostationary Transfer Orbits (GTO), likely due to the higher fuel reserves available for the landing burn.
- **Booster Version Reliability:** Data queried via SQL and visualized in the dashboard highlights that newer booster versions (especially "FT" and "B5") have significantly higher success rates than the earlier "v1.1" versions. This confirms that hardware iterations have directly addressed previous failure points.

Conclusions

- **High Predictive Accuracy:** The machine learning models achieved a consistent accuracy of approximately **83.3% on test data**. This proves that booster landing success is not random but follows detectable patterns that can be quantified using variables like payload mass, launch site, and orbit type.
- **Consistency Across Algorithms:** During the ML phase, Logistic Regression, SVM, and KNN models yielded similar results (~84.8% validation accuracy). However, the **Decision Tree** model slightly outperformed them during cross-validation (87.6%), suggesting it was better at capturing specific non-linear relationships in the dataset.
- **Economic Impact of Landing Success:** By accurately predicting landing outcomes, we can estimate launch costs more effectively. Since the first stage represents a major portion of the total rocket cost, successful recovery is the primary driver behind SpaceX's ability to offer lower market prices compared to competitors.

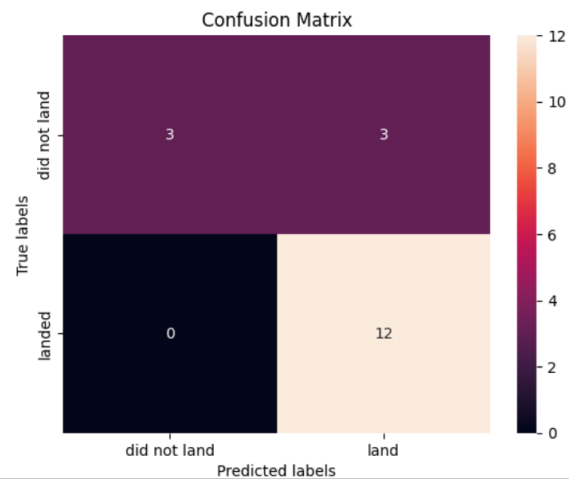
Appendix

```
[32]: test_accuracy = logreg_cv.score(X_test, Y_test)
      print("Test accuracy:", test_accuracy)
```

Test accuracy: 0.8333333333333334

Lets look at the confusion matrix:

```
[33]: yhat=logreg_cv.predict(X_test)
      plot_confusion_matrix(Y_test,yhat)
```



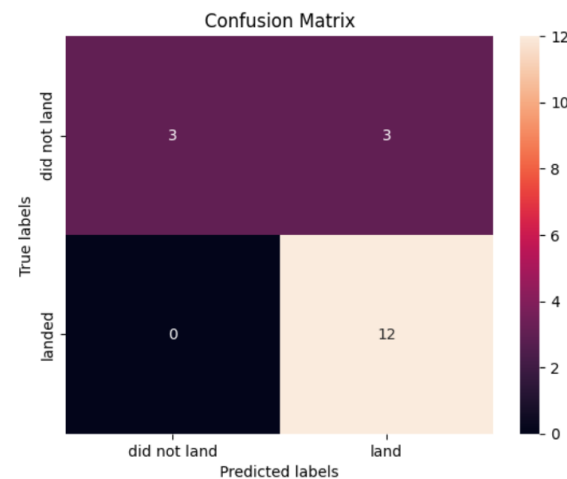
Confusion Matrix of Logistic Regression

```
[40]: test_accuracy = svm_cv.score(X_test, Y_test)
      print("Test accuracy:", test_accuracy)
```

Test accuracy: 0.8333333333333334

We can plot the confusion matrix

```
[41]: yhat=svm_cv.predict(X_test)
      plot_confusion_matrix(Y_test,yhat)
```



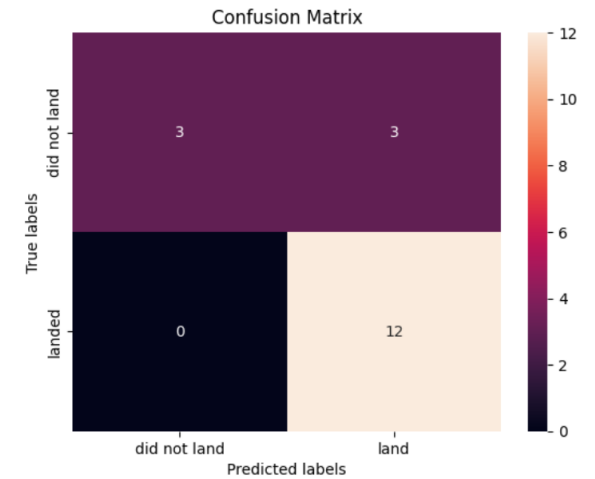
Confusion Matrix of Support Vector Machine (SVM)

```
[57]: test_accuracy = knn_cv.score(X_test, Y_test)
      print("Test accuracy:", test_accuracy)
```

Test accuracy: 0.8333333333333334

We can plot the confusion matrix

```
[58]: yhat = knn_cv.predict(X_test)
      plot_confusion_matrix(Y_test,yhat)
```



Confusion Matrix of K-Nearest Neighbors

Thank you!

